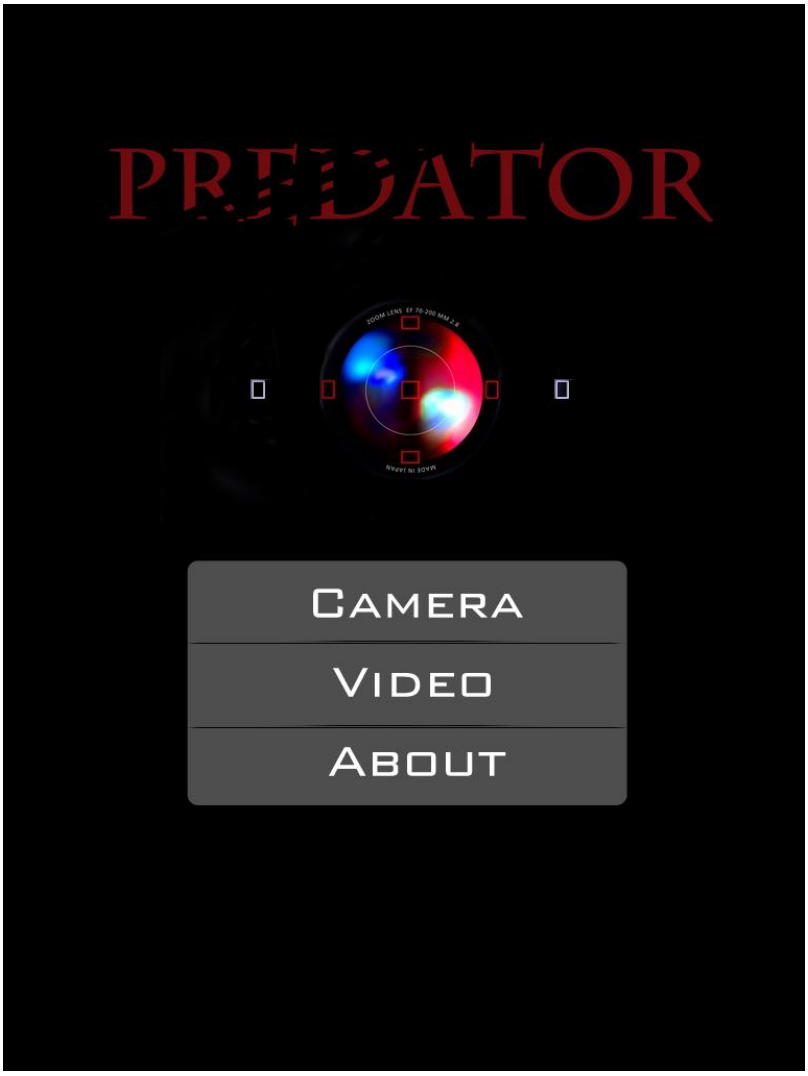
Predator - Tracking, Learning and Detection



Project by:
Majd Srour
Marian Mansour
Under the supervision of:
Aaron Wetzler and Yaron Honen

Contents

Predator - Tracking, Learning and Detection 1
Abstract 3
Introduction 3
More Details 4
  Problem Description 4
  Solution 4
    Solution in a nutshell 4
    Framework 4
    System pipeline 8
Results 9
Conclusion 10
Acknowledgment 10
References 11

Abstract

A key function in computer vision is object tracking. Many computer vision fields rely on object tracking to function correctly. Fields such as human-computer interaction, security and surveillance, video communication and compression, augmented reality, traffic control, medical imaging and video editing [1] use such algorithms. Therefore, Having a reliable tracking algorithm is necessity. Combining the importance of computer vision, and the fast growing market of *Mobile* phones with a tracking algorithm powerful enough to give us reliable tracking yet efficient enough to work on low powered CPU (*Mobile Phone*) can significantly benefit existing and future mobile phone infrastructure.

Introduction

Predator (T.L.D) is a real-time algorithm for tracking of previously unseen objects in video streams.- The object of interest is defined by a bounding box in the first frame. The result is real-time tracking of the object that improves over time. The algorithm was developed in Matlab and C++ by Zdenek Kalal during his PhD thesis under the supervision of Krystian Mikolajczyk and Jiri Matas. The purpose of our project was to fully re-implement the *Predator* algorithm for real time operation on an iPhone/iPad.

More Details

Problem Description

Given an initial image (first frame), the bounding box selected and consecutive frames of the video (camera), the task is to output a bounding box surrounding the object of interest on each frame.

Solution

Solution in a nutshell

We train 2 Classes from the first image:

- Positive Class: patches inside the bounding box (object of interest)
- Negative Class: patches in the surrounding background.

For each frame, we scan for similar patches over many positions and scales and then select the patch with the higher confidence.

Framework

Four major components:

- iPhone SDK (Camera APIs)
- Tracker
- Detector
- Learner

iPhone SDK

Using the iPhone/iPad SDK and the Camera API, we access the camera, taking each frame, and convert it to a *Grayscale* image using NEON Assembly primitives. Doing so guarantees close-to-the-metal operation as a result of utilizing the device's parallelized SIMD operations. The SDK is broken down into 4 sets: Cocoa Touch, Media and Core Services. We used Cocoa Touch that includes:

1. Multi-touch events and controls
2. View hierarchy
3. Camera support

Our code was written in C++ and Objective-C, we wrote the core of the tracking algorithm in C++ in order to gain performance, and the GUI part was written in Objective-C.

Tracker

The tracker takes two images, bounding box on the 1st image and outputs bounding box on the second image

Algorithm:

1. Select a grid of points in the first image inside the bounding box
2. Track those points on the 2nd image using the Lucas-Kanade tracker provided in OpenCV
3. Filter points with low tracking confidence
4. Track the remaining points backward with Lucas-Kanade
5. Re-filter the remaining points with low tracking confidence

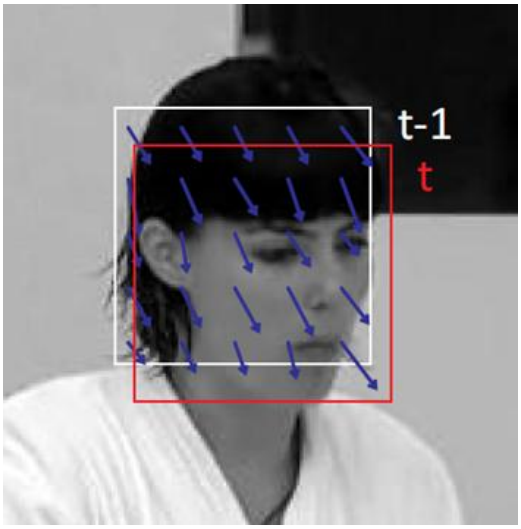


Figure 1. Tracking the bounding box on the 2nd image[4]

Now we have set of points which are Forward-Backward consistent. These points are then used to estimate the displacement of the bounding box and scale change.

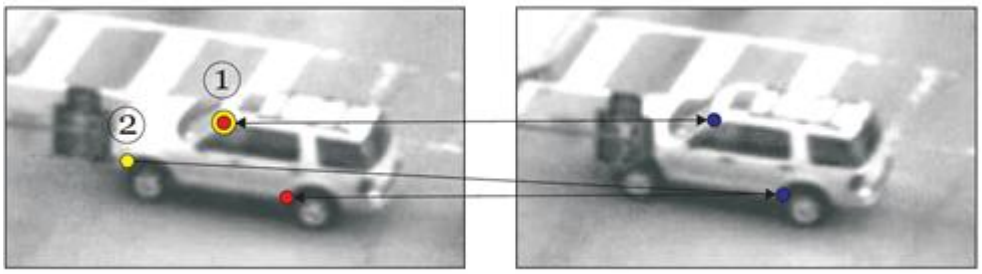


Figure 2. Forward-Backward tracking using Lucas-Kanade[4]

Displacement calculation:

1. For each pair of remaining points calculate (dx,dy) - tracking point displacement
2. bounding box displacement : (median(dx), median(dy))

Scale change calculation:

1. For each pair of remaining points, we calculate the ratio between the distance in the 2nd image and 1st image.
2. Scale change is the median of all those ratios.

After having the displacement and scale, we can test using Normalized Cross Correlation whether those two patches are similar or not.

Detector

The detector takes an image and then outputs candidate bounding boxes which have the highest confidence.
We used a sliding window approach, scanning the entire frame with overlapping bounding-boxes at several scales and classifying each patch using an optimized Fern Classifier.
If a detected patch overlaps the tracked patch by more than some threshold it is considered that the detected patch must be positive if and only if we are confident that the tracked patch tightly contains the object we are tracking.

Fern Classifier

The Predator classifier is a random forest classifier consisting of random ferns. We have a slightly special case in that there are only two classes (positive and negative).

Random Ferns

Random ferns are similar to random trees except that all patches are evaluated at every node. Instead of using a tree we have an ordered set of nodes. Each node contains a test on the patch: a feature. We describe the result of the test as a number in binary representation.. Accumulating, in order, the results of all the nodes computed on a single patch, defines a binary code that specifies a specific leaf node.
For the two-class case, each leaf node records the number of positive p and negative n examples training examples that fall into it. The posterior then is computed by maximum likelihood estimator $P(c = 1) = \frac{p}{(p+n)}$ [3] or 0 if $p = 0$[3] where c is the class of the patch and is 1 if positive. In the case where we have more than 2 classes, the posterior is computed as $P(c = X) = \frac{x}{n}$[3] or 0 if $x = 0$[3], where X is the class we are computing the posterior for, x is the number of class X training examples that have fallen into this leaf, and n is the total number of training samples that have fallen into this patch. Note that while an initial training phase is required, this classifier can be updated with new samples at any time.

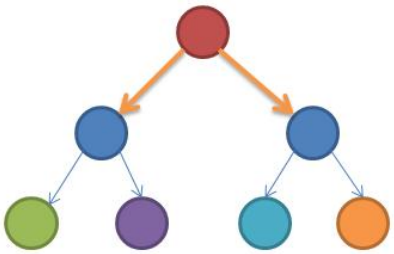


Figure 3 – Fern tree

- Each non-terminal node contains a simple test (Feature)
- Each leaf contains an estimate of positive images which reach it (positive samples/ total samples)
- Features (tests) are randomly chosen, and multiple trees make up the whole forest
- The tree contains the same test at each level of the tree
- Forest estimation = $\frac{1}{n}\sum_{i=1}^{n} TreeEstimation_i$

Features

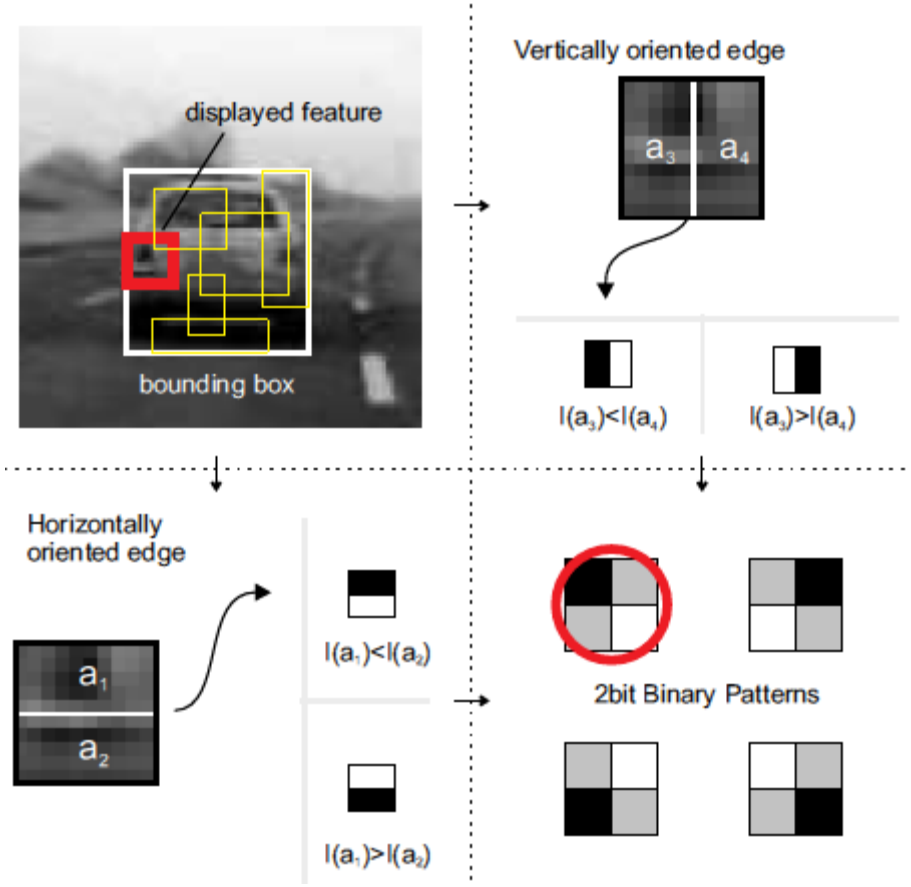


Figure 4. 2bit Binary Patterns used in our implementation, Features encode local gradient orientation[3]

The features we used are called 2Bit binary patterns. These features measure gradient orientation within a certain area, quantize it and output four possible codes (see Fig. 4). We divide the area vertically and horizontally and then test which part is brighter, as we see in the figure 4, the first bit is defined by area 3 vs. area 4, and the 2nd bit is defined by area1 vs. area2
The output can be a 4 possible numbers (0, 1, 2, and 3) which are 2 bits.

System pipeline

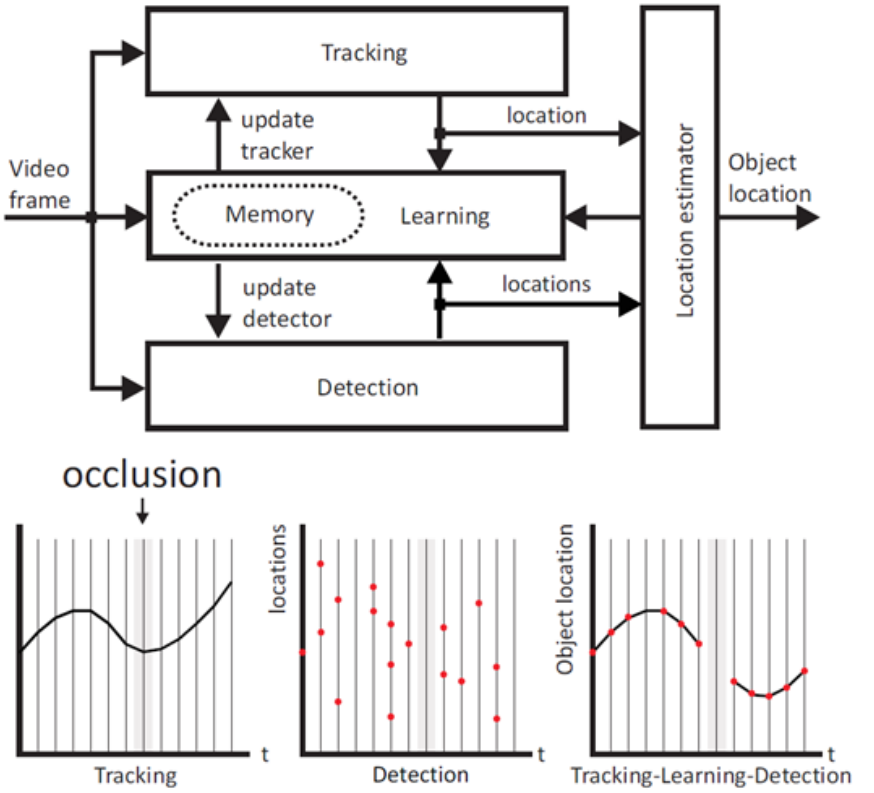


Figure 5. Integration of all the parts together[2]

Integrating all the parts together gives us the above picture.
for each frame:

1. The tracker returns the tracked patch
2. The detector returns detected patches (unlabeled patches, to be classified later)
3. If the Normalized-Cross-Correlation of the previously detected patch with the original patch is higher than a threshold (0.7 in our case) and the tracked patch has higher confidence than all detected patches then we apply the Positive-Negative learning process. If not then we discard all detected patches
    a. Application of P constraints: samples labeled as Negative that are spatially close to the tracked patch
    b. Application of N constraints: samples labeled as Positive that are spatially distant from the tracked patch
4. The classifier is retrained on the training set according to the classification above. In practice, we only require updating the p and n variables in the leaf nodes the new patches fall into in order to update the posteriors

Results

We built an application on an iPhone 4 and iPad for tracking of unknown objects in real-time manner.
The timing results of the tracking were quite surprising, as the original code written by Zdenek Kalal, was run at ~15fps (on a Core2 Duo system) while our iPhone code runs only slightly slower at ~9fps on an iPad (low-power CPU).

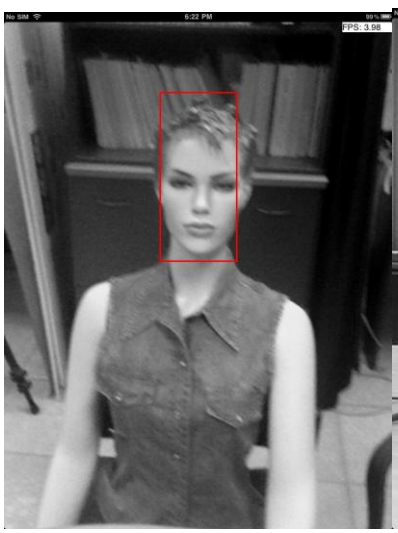

Figure 5.1


Figure 5.2

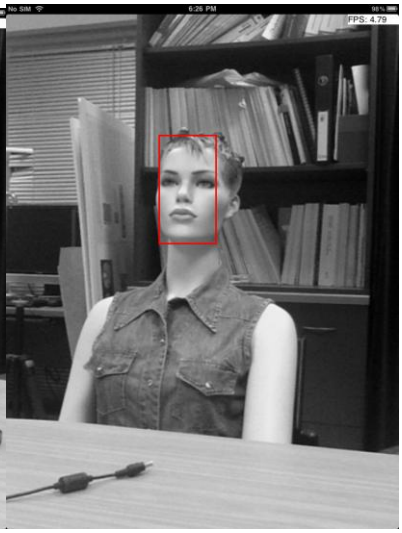

Figure 5.3

Figure 5.1: Initializing the application with a bounding box around the face (object of interest)
Figure 5.2: Moving away from the tracked object with small rotation
Figure 5.3: Getting closer to the object.

Conclusion

The algorithm consists of two parts: the tracking component and the detection component. While doing our testing we noticed positive and negative behavior of the algorithm, in some cases it tracks the object without losing it, in other cases it loses the object. Those observations of the algorithm can be summarized as the following:

Positive:

- Tracks successfully from the first frame
- Learns light transformation - small rotations, translations
- Successfully detects the object if it disappeared.
- Handles scaling successfully, when the object come closer or drifting from the camera.

Negative:

- Very sensitive to fast movement, tracks better on slow movements.
- Sensitive to things that look similar.
- Fails to detect when the bounding box around the object is too small (lack of features)
- Sensitive to light conditions (though, we tried to fix it by normalizing the image and blurring it with a Gaussian filter - and now it's much less sensitive)

Overall, we think our implementation was a success, though there's still place for improvements, the algorithm is better at tracking the object than detecting. And it tracks some objects (those with more details) much better than others.

Acknowledgment

We would like to thank Professor Ron Kimmel for providing us with the necessary equipment and *Yaron Honen* for helping with lab administration.

We would also like to thank *Aaron Wetzler* for the great lectures he gave us and the time spent helping us reaching our goal - a BIG Thank you!

Lastly, we'd like to thank *Zdenek Kalal*, for developing this award-winning outstanding algorithm, and for his contribution to the open source community by sharing the *Matlab* source code of *TLD* (openTLD) - Thank you!

References

1. “Video Tracking”
http://en.wikipedia.org/wiki/Video_tracking
2. Z. Kalal, J. Matas, and K. Mikolajczyk, “Online learning of robust object detectors during unstable tracking,” *On-line Learning for Computer Vision Workshop*, 2009.
info.ee.surrey.ac.uk/Personal/Z.Kalal/Publications/2009_olcv.pdf
3. Z. Kalal, J. Matas, and K. Mikolajczyk, “P-N Learning: Bootstrapping Binary Classifiers by Structural Constraints,” *Conference on Computer Vision and Pattern Recognition*, 2010.
info.ee.surrey.ac.uk/Personal/Z.Kalal/Publications/2010_cvpr.pdf
4. Z. Kalal, K. Mikolajczyk, and J. Matas, “Forward-Backward Error: Automatic Detection of Tracking Failures,” *International Conference on Pattern Recognition*, 2010, pp. 23-26.
info.ee.surrey.ac.uk/Personal/Z.Kalal/Publications/2010_icpr.pdf
5.M. Özuysal, M. Calonder, V. Lepetit, P. Fua “Fast Keypoint Recognition using Random Ferns” IEEE Transactions on Pattern Analysis and Machine Intelligence, Vol. 32, Nr. 3, pp. 448 - 461, March 2010
cvlab.epfl.ch/~lepetit/papers/ozuysal_pami10.pdf